

Bigdata 101

HDFS Cluster Kurulum ve Yarn
Yapılandırma

Veysel Yüksel



Table of Contents

01

HDFS Cluster Kurulumu

02

HDFS Servis Yönetimi

03

Yarn Capacity ve Queue Kavramları

04

Mapreduce Örnek

05

YARN Resource Manager UI

06

YARN Queue Oluşturma

HDFS Yapılandırma Dosyaları

Dosya Adı	Açıklama
core-site.xml	Hadoop çekirdeği için genel ayarları (örneğin NameNode URI, temporary directory) içerir.
hdfs-site.xml	HDFS (Hadoop Distributed File System) ile ilgili ayarları (örneğin replication, permission, fsimage konumu) tanımlar.
mapred-site.xml	MapReduce framework'ü ile ilgili ayarları (örneğin job tracker, execution framework) yapılandırır.
yarn-site.xml	YARN (Yet Another Resource Negotiator) ile ilgili kaynak yöneticisi ve scheduler ayarlarını içerir.
<u>hadoop-env.sh</u>	Hadoop süreçleri için çevresel değişkenleri (örneğin JAVA_HOME, HADOOP_HEAPSIZE) tanımlar.
<u>mapred-env.sh</u>	MapReduce çalıştırmaları için çevresel değişkenleri tanımlar.
<u>yarn-env.sh</u>	YARN bileşenleri (ResourceManager, NodeManager) için çevresel değişkenleri içerir.
slaves (veya workers)	Hadoop cluster'daki worker (DataNode, NodeManager) sunucularının hostname/IP listesini içerir.
masters	Secondary NameNode veya master servislerinin çalışacağı sunucuları belirtir (genellikle deprecated).
capacity-scheduler.xml	YARN kapasite planlayıcısına özgü queue'ların yapılandırmasını içerir.
container-executor.cfg	YARN'ın LinuxContainerExecutor kullanırken uyguladığı güvenlik politikalarını tanımlar.
log4j.properties (veya log4j2.properties)	Hadoop bileşenleri için loglama seviyelerini ve log çıktılarının konumlarını yapılandırır.

HDFS Yapılandırma Dosyaları

fsimage ve editlog gibi metadata'lar doğrudan Java heap'e yüklenir. Bu nedenle NameNode'un heap size'ı yeterince büyük olmalıdır.

- Sadece fiziksel RAM'in fazla olması yeterli değildir — JVM heap olarak kullanmazsa işe yaramaz.
- Eğer heap yetersizse:
 - java.lang.OutOfMemoryError: Java heap space
 - Veya JVM NameNode'u hiç başlatamaz (sessiz crash olur)
 - Büyük HDFS (milyonlarca dosya) için bu çok kritik hale gelir
- HDFS Objeleri (dosya, blok) Tahmini Heap Gereksinimi
 - 1 milyon dosya + blok ~1.5 GB heap
 - 10 milyon dosya + blok ~10–16 GB heap
 - 100 milyon dosya + blok 50+ GB heap
- NN heap size Hadoop-env.sh içinde aşağıdaki gibi ayarlanabilir;
- `export HADOOP_NAMENODE_OPTS="-Xms16g -Xmx16g $HADOOP_NAMENODE_OPTS"`
- fiziksel memory'nin max %50'si NN JVM heap için ayrılabilir.

HDFS Federation

Eğer tek bir NameNode ile JVM heap sınırları aşıyor ise (örneğin yüz milyonlarca dosya + blok varsa);

- Federation (HDFS Federation) kullanarak birden fazla bağımsız NameNode
- veya HDFS'yi fiziksel olarak ayrı cluster'lara ayırmak

HDFS Federation

- Her biri kendi NameNode + DataNode setine sahip bağımsız namespace'ler.
- Aynı DataNode fiziksel olarak birden fazla NameNode'a hizmet verebilir.
- Ne zaman düşünülmeli?
 - Heap > 60–80 GB çıkıyorsa
 - Milyarlarca dosya ve blok varsa
 - Metadata latency'si hissedilir derecede artıyorsa
 - GC zamanları > 2-3 saniye sürüyorsa

HDFS Federation

Eğer tek bir NameNode ile JVM heap sınırları aşıyor ise (örneğin yüz milyonlarca dosya + blok varsa);

- Federation (HDFS Federation) kullanarak birden fazla bağımsız NameNode
- veya HDFS'yi fiziksel olarak ayrı cluster'lara ayırmak

HDFS Federation

- Her biri kendi NameNode + DataNode setine sahip bağımsız namespace'ler.
- Aynı DataNode fiziksel olarak birden fazla NameNode'a hizmet verebilir.
- Ne zaman düşünülmeli?
 - Heap > 60–80 GB çıkıyorsa
 - Milyarlarca dosya ve blok varsa
 - Metadata latency'si hissedilir derecede artıyorsa
 - GC zamanları > 2-3 saniye sürüyorsa

HDFS Federation

1. LinkedIn

- Petabaytlarca veri, yüz milyonlarca dosya.
- Federation ile farklı kullanım alanlarını (örneğin: ETL, real-time, archival) ayrı NameNode'lara bölüyorlar.
- Kendi HDFS'lerini daha da ölçeklenebilir hâle getirmek için Federation + YARN kombinasyonu kullanıyorlar.

2. Yahoo (Verizon Media)

- İlk katkısı yapanlardan biri.
- Onlarca farklı namespace ile HDFS Federation'ı uzun süredir kullanıyorlar.
- 10+ petabyte üzerinde çalışıyorlar.

3. Spotify

- Özellikle DataLake üzerindeki yoğun I/O'yu bölmek ve izole etmek için Federation yapısı kuruldu.
- Farklı veri hatlarını farklı namespace'lere yönlendiriyorlar (örneğin: analytics vs. raw ingestion).

4. Netflix

- Devasa video analizleri, log sistemleri ve farklı veri yaşam döngüleri için Federation kullandılar.
- Her NameNode farklı bir kullanım amacına hizmet ediyor (örneğin: iş zekası vs. stream processing).

HDFS Federation

1. LinkedIn

- Petabaytlarca veri, yüz milyonlarca dosya.
- Federation ile farklı kullanım alanlarını (örneğin: ETL, real-time, archival) ayrı NameNode'lara bölüyorlar.
- Kendi HDFS'lerini daha da ölçeklenebilir hâle getirmek için Federation + YARN kombinasyonu kullanıyorlar.

2. Yahoo (Verizon Media)

- İlk katkıcı yapanlardan biri.
- Onlarca farklı namespace ile HDFS Federation'ı uzun süredir kullanıyorlar.
- 10+ petabyte üzerinde çalışıyorlar.

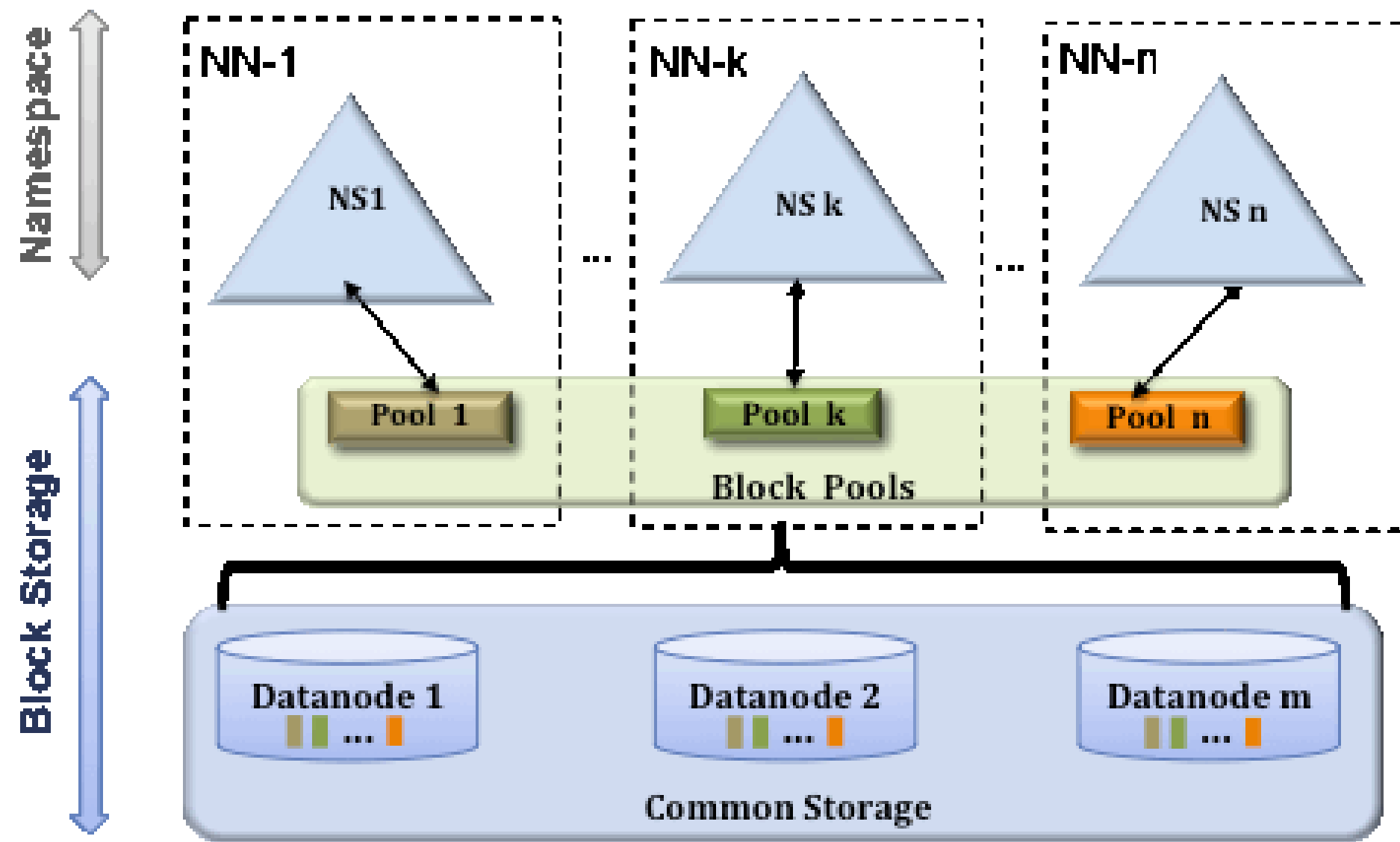
3. Spotify

- Özellikle DataLake üzerindeki yoğun I/O'yu bölmek ve izole etmek için Federation yapısı kuruldu.
- Farklı veri hatlarını farklı namespace'lere yönlendiriyorlar (örneğin: analytics vs. raw ingestion).

4. Netflix

- Devasa video analizleri, log sistemleri ve farklı veri yaşam döngüleri için Federation kullandılar.
- Her NameNode farklı bir kullanım amacına hizmet ediyor (örneğin: iş zekası vs. stream processing).

HDFS Federation



```
<configuration>
  <property>
    <name>dfs.nameservices</name>
    <value>ns1,ns2</value>
  </property>
  <property>
    <name>dfs.namenode.rpc-address.ns1</name>
    <value>nn-host1:rpc-port</value>
  </property>
  <property>
    <name>dfs.namenode.http-address.ns1</name>
    <value>nn-host1:http-port</value>
  </property>
  <property>
    <name>dfs.namenode.secondary.http-address.ns1</name>
    <value>snn-host1:http-port</value>
  </property>
  <property>
    <name>dfs.namenode.rpc-address.ns2</name>
    <value>nn-host2:rpc-port</value>
  </property>
  <property>
    <name>dfs.namenode.http-address.ns2</name>
    <value>nn-host2:http-port</value>
  </property>
  <property>
    <name>dfs.namenode.secondary.http-address.ns2</name>
    <value>snn-host2:http-port</value>
  </property>

  .... Other common configuration ...
</configuration>
```

dfs.nameservices : Bu parametre, kümede kaç adet namespace kullanılacağını belirtir.

dfs.namenode.rpc-address : Namespace hangi NN'a bağlanacak

dfs.namenode.http-address.ns1 : Http web ui

dfs.namenode.secondary.http-address.ns1 : secondary nn web ui

Yarn Scheduler Nedir?

Scheduler kaynakların yönetimini ve dağıtılmasını sağlayan bileşendir. İşlerin zamanında çalışabilmesi için kaynak kullanımlarını planlar, dağıtır ve bu kaynakları izler. Scheduler'ın başlıca görevleri;

- Kaynakların dağıtılması,
- İşlerin planlanması,
- Kuyruk(Queue) yönetimi,
- Cluster üzerinde iş yükünün dengeli dağıtılması,
- Kaynakların dağıtılırken adil ve eşit biçimde dağıtılması,
- Kaynakların kullanım durumları hakkında metriklerin toplanması.

YARN Scheduler

Scheduler kaynak dağıtma işlemi göz önüne alındığında, tercih edilebilecek 2 farklı method vardır.

1. **Capacity Scheduler** : Kaynak dağıtım işlemi oluşturulan kuyrukların kapasitesine göre yapılır. Yöneticiler tarafından farklı işler için farklı kapasitelere sahip kuyruklar oluşturulur ve çalışan iş kaynaklarını o kuyruk kapasinde tanımlandığı ölçülerde kullanır.
2. **Fair Scheduler** : Kaynakların çalışan işler arasında eşit olarak paylaşılmasını hedefler, böylece uygulamalar birbirinin önüne geçmez ve her uygulama yeterli kaynağı alabilir.

YARN Queue Kavramı

Queue Kavramı : İşlerin kaynak kullanımını organize etmenin bir yöntemidir. Kuyruklar kaynakların kullanımı için bir grup uygulamanın kullandığı mantıksal yapılardır denebilir.

Kuyruklar aracılığı ile yapılabilecekler;

- Kaynak yönetimi,
- Yük dengeleme,
- Ve önceliklendirme.
- Dolayısı ile bir kuyruğun kapasite, öncelik bilgisi ve paylaşım gibi 3 temel özelliği bulunur.
- capacity-scheduler.xml ve fair-scheduler.xml dosyaları aracılığı ile konfigürasyonlar belirlenir.

YARN Parametreler

yarn.scheduler.minimum-allocation-mb: YARN Scheduler'ın bir uygulama için tahsis edebileceği minimum bellek miktarını belirler. Resource Manager ile ilgili bir parametredir.

yarn.scheduler.maximum-allocation-mb: YARN Scheduler'ın bir uygulama için tahsis edebileceği maksimum bellek miktarını belirler. Resource Manager ile ilgili bir parametredir.

yarn.nodemanager.resource.memory-mb: Her NodeManager'ın tahsis edebileceği toplam bellek miktarını belirler. Nodemanager'lar ile ilgili bir parametredir.

yarn-site.xml dosya örneği;

```
<configuration>
  <property>
    <name>yarn.scheduler.minimum-allocation-mb</name>
    <value>1024</value>
  </property>
  <property>
    <name>yarn.scheduler.maximum-allocation-mb</name>
    <value>2048</value>
  </property>
  <property>
    <name>yarn.nodemanager.resource.memory-mb</name>
    <value>4096</value>
  </property>
</configuration>
```


Mapreduce - Parametreler

Kurulumdan sonra mapreduce işlerinin kullanabileceği kaynakları da mapred-site.xml dosyası içerisinde ayarlanabilir.
Bir uygulamanın map ve reduce aşamalarında kullanabileceği kaynak değerleri belirlenir.

```
<configuration>
  <property>
    <name>mapreduce.map.memory.mb</name>
    <value>2048</value> <!-- 2 GB -->
  </property>
  <property>
    <name>mapreduce.reduce.memory.mb</name>
    <value>4096</value> <!-- 4 GB -->
  </property>
  <property>
    <name>mapreduce.map.cpu-vcores</name>
    <value>1</value>
  </property>
  <property>
    <name>mapreduce.reduce.cpu-vcores</name>
    <value>2</value>
  </property>
</configuration>
```

YARN Resource manager UI

- Bu dosyalar editlendikten sonra slavenode'larda da değişiklikler yapılır ve cluster servisleri stop start edilerek
- ResourceManager(8088) - NodeManager(8042) web ui'lerinden kaynak kontrolü yapılabilir.

oop

Nodes of the cluster

Logged in as:

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources	Total Resources	Reserved Resources	Physical Mem Used %	Physical VCores %
0	0	0	0	0	<memory:0 B, vCores:0>	<memory:8.56 GB, vCores:16>	<memory:0 B, vCores:0>	21	0

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes	Shutdown Nodes
2	0	0	0	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation	Maximum Cluster Application Priority	Scheduler Busy %	RM Dispatcher EventQueue Size	Scheduler Dispatcher EventQueue Size
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:2192, vCores:4>	0	0	0	0

Show 20 entries

Node Labels	Rack	Node State	Node Address	Node HTTP Address	Last health-update	Health-report	Containers	Allocation Tags	Mem Used	Mem Avail	Phys Mem Used %	VCores Used	VCores Avail	Phys VCores Used %	Version
	/default-rack	RUNNING	slavenode02.us-central1-a.c.praxis-life-428819-h1.internal:46801	slavenode02.us-central1-a.c.praxis-life-428819-h1.internal:8042	Wed Sep 04 19:14:50 +0000 2024		0		0 B	4.28 GB	21	0	8	0	3.4.0
	/default-rack	RUNNING	slavenode01.us-central1-a.c.praxis-life-428819-h1.internal:34261	slavenode01.us-central1-a.c.praxis-life-428819-h1.internal:8042	Wed Sep 04 19:14:50 +0000 2024		0		0 B	4.28 GB	21	0	8	0	3.4.0

Showing 1 to 2 of 2 entries

First Previous 1 Next

Örnek Uygulama



- Hadoop clusterda tanımlı gelen wordcount örneğini çalıştırıp cluster'ın durumunu inceleyelim.
- Hadoopta örnek olarak çalıştıracağımız uygulama kurulumdan sonra `/cluster/hadoop/share/hadoop/mapreduce` dizini altında bulunan `hadoop-mapreduce-examples-3.4.0.jar` uygulamasıdır. Bu uygulama wordcount uygulamasıdır. Input olarak verilen hdfs üzerindeki bir dosyayı alarak output olarak bu dosyada yer alan kelime sayısını döndürür. Uygulamayı çalıştırmak için öncelikle bir metin dosyası oluşturarak hdfs'e kopyalamak gerekir.

Resource Manager Web UI

Giriş ekranında cluster metrikleri görüntülenebilir;
Toplam submit edilen iş sayısı, çalışan iş sayısı, pending statüsündeki iş sayısı ve cluster'ın sahip olduğu toplam kaynaklar Cluster metrics bölümünde görüntülenebilir.



Cluster

About
Nodes
Node Labels
Applications
NEW
NEW SAVING
SUBMITTED
ACCEPTED
RUNNING
FINISHED
FAILED
KILLED
Scheduler

Tools

Logged in as: dr.who

About the Cluster

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources	Total Resources	Reserved Resources	Physical Mem Used %	Physical VCores Used %
3	0	0	3	0	<memory:0 B, vCores:0>	<memory:8 GB, vCores:16>	<memory:0 B, vCores:0>	37	0

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes	Shutdown Nodes
2	0	0	0	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation	Maximum Cluster Application Priority	Scheduler Busy %	RM Dispatcher EventQueue Size	Scheduler Dispatcher EventQueue Size
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:2048, vCores:4>	0	0	0	0

Cluster overview

Cluster ID:

1752750013831

ResourceManager state:

STARTED

ResourceManager HA state:

active

ResourceManager HA zookeeper connection state:

Could not find leader elector. Verify both HA and automatic failover are enabled.

ResourceManager RMStateStore:

org.apache.hadoop.yarn.server.resourcemanager.recovery.NullRMStateStore

ResourceManager started on:

Thu Jul 17 11:00:13 +0000 2025

ResourceManager version:

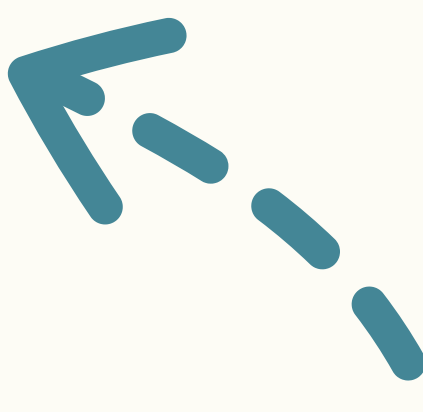
3.4.0 from bd8b77f398f626bb7791783192ee7a5dfaee760 by root source checksum 934da0c5743762b7851cfcff9f8ca2 on 2024-03-04T06:54Z

Hadoop version:

3.4.0 from bd8b77f398f626bb7791783192ee7a5dfaee760 by root source checksum f7fe694a3613358b38812ae9c31114e on 2024-03-04T06:35Z



Resource Manager Web UI

- **Vcore ve vmemory** kavramları yarn'ın kaynak yönetimi sırasında fiziksel kaynaklar için oluşturduğu sanal iz düşümleridir. Bazı durumlarda fiziksel bir kaynak virtual birden çok kaynak tarafından kullanılabilir.
 - **Cluster Nodes metrics alanında**, nodemanager'larla alakalı metrikler yer alır. Clusterde şu anda active 2 node manager vardır. Nodemanager servislerinde bir sorun olduğunda lost nodes veya unhealthy node sayıları sıfırdan farklı olur.
 - **Scheduler metrics** alanında kaynakların yönetimi ile ilgili metrikler görülür. Şu anda capacity scheduler kullanılmaktaydı. Oluşturulan farklı kuyruklar ve özellikleri de bu alanda görüntülenebilir.
 - **En altta Cluster overview** alanında ise ClusterID, resource manager'ın statusu, Resource manager'ın başlatılma saati ve versiyonu, hadoop cluster'ın versiyonu gibi bilgiler yer almaktadır.
- 

YARN Application Status

Yarn'da bir application'un durumları aşağıdakilerden biri olabilir;

- 1. **NEW** : YARN Scheduler'a gönderilmiştir ancak henüz kaynak tahsisi veya başlatma işlemleri yapılmamıştır. Bu aşamada uygulama, sistem tarafından henüz işleme alınmamıştır. Yeni oluşturulan bir uygulama ilk olarak bu durumda bulunur.
- 2. **NEW_SAVING**: Uygulama yeni oluşturulmuş ve kaynak talepleri kaydedilmiştir, ancak kaynak tahsisi veya çalıştırma işlemleri başlamamıştır. Bu durumda uygulama, kayıt aşamasındadır.
- 3. **SUBMITTED** : Uygulama Scheduler tarafından kabul edilmiştir fakat henüz kaynak tahsisi yapılmamıştır. Bu durumda, uygulamanın çalıştırılması için kaynak tahsisi beklenmektedir.
- 4. **ACCEPTED** : Uygulama, Scheduler tarafından kabul edilmiş ve kaynak tahsis edilmesi için sıralamaya alınmıştır. Bu durumda, uygulamanın kaynak tahsisi yapılmak üzeredir.
- 5. **RUNNING** : Uygulama, kaynak tahsisi yapılmış ve çalıştırılmakta olan bir durumdadır. Bu aşamada, uygulamanın iş parçacıkları veya görevleri aktif olarak çalışmaktadır.
- 6. **FINISHED** : Uygulama başarıyla tamamlanmıştır. Bu aşamada, uygulamanın tüm görevleri başarıyla yürütülmüş ve tamamlanmış demektir.
- 7. **FAILED** : Uygulama, çalıştırılma sırasında hata almış veya başarısız olmuştur. Bu durumda, uygulama işini tamamlayamaz ve hata raporlarıyla birlikte başarısız sayılır.
- 8. **KILLED** : Uygulama, kullanıcının veya sistem yöneticisinin müdahalesi ile manuel olarak sonlandırılmıştır. Bu durumda, uygulama, başlatılmadan veya çalıştırılmadan önce veya çalışma sırasında sonlandırılmış olabilir.
- 9. **DELETED** : Uygulama, tamamen silinmiş ve sistemden kaldırılmıştır. Bu durumda, uygulamanın verileri ve kaynakları kalıcı olarak silinmiş olabilir.

YARN Container

Bir container, belirli miktarda kaynak tahsis etmiş ve bir işin çalıştırılabilir bir ortamını temsil eden sanal bir birimdir. YARN'da işlerin izole bir şekilde çalıştırılmasını sağlar.

Container'ın çalışma adımları şunlardır;

1. Uygulama İsteği: Bir uygulama çalıştırmak için, uygulama belirli bir kaynak talebi (bellek, CPU) ile ResourceManager'a başvurur.
2. Kaynak Tahsisi: ResourceManager, bu talebi alır ve uygun kaynakları tahsis edebilecek bir NodeManager bulur. NodeManager, uygun container'ı oluşturur ve gerekli kaynakları tahsis eder.
3. Container Başlatma: NodeManager, tahsis edilen kaynaklarla container'ı başlatır. Container içerisinde uygulamanın iş yükü çalıştırılır.
4. Kapsayıcı Yönetimi: Uygulamanın çalışması sırasında container, kaynak kullanımını izler ve yönetir. Uygulama tamamlandığında, container kapanır ve kaynaklar serbest bırakılır.

YARN Application Master

Her uygulama'nın yaşam döngüsünü yöneten bir Application Master'ı bulunur. Uygulamaya dair yönetimsel ve çalışma süreçlerinin yönetiminden sorumludur. AM, resourcemanager'dan iş'in ihtiyaç duyduğu kaynakları talep eder ve işi çalıştırır. Container'ları talep ederek, iş yükünü bu container'lar üzerine dağıtılmasını sağlar. İşin çalışmasını izleyerek başarısız olması durumunda yeniden denemeyi yapar. Uygulama tamamlandığında veya hata aldığında gerekli temizlik işlemlerini AM yapar. Her Application Master genellikle kendi web arayüzüne sahiptir, bu da uygulamanın ilerleyişi ve durumunun izlenmesini sağlar. Genellikle şu URL ile erişilebilir: `http://<node-manager-host>:<am-port>`

YARN New Queue

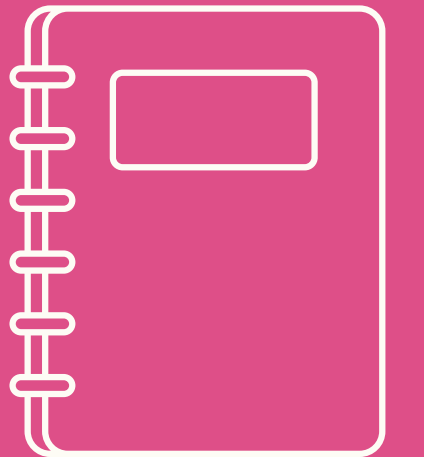
YARN CapacityScheduler'da kuyruk isimlendirmeleri hiyerarşik bir yapıdadır.

En üst seviyede bir "root" kuyruğu bulunur. Bu kök kuyruk, tüm alt kuyrukların ve yapıların temelidir.

Root kuyruk altında bir veya daha fazla alt kuyruk olabilir. Bu alt kuyruklar, belirli uygulamalar veya iş yükleri için kaynakları daha detaylı bir şekilde yönetmek amacıyla kullanılır.

Alt kuyrukların isimleri genellikle `yarn.scheduler.capacity.root.`

`<kuyruk_adi>` formatında belirlenir. Örneğin, `yarn.scheduler.capacity.root.default` varsayılan bir kuyruğu temsil eder



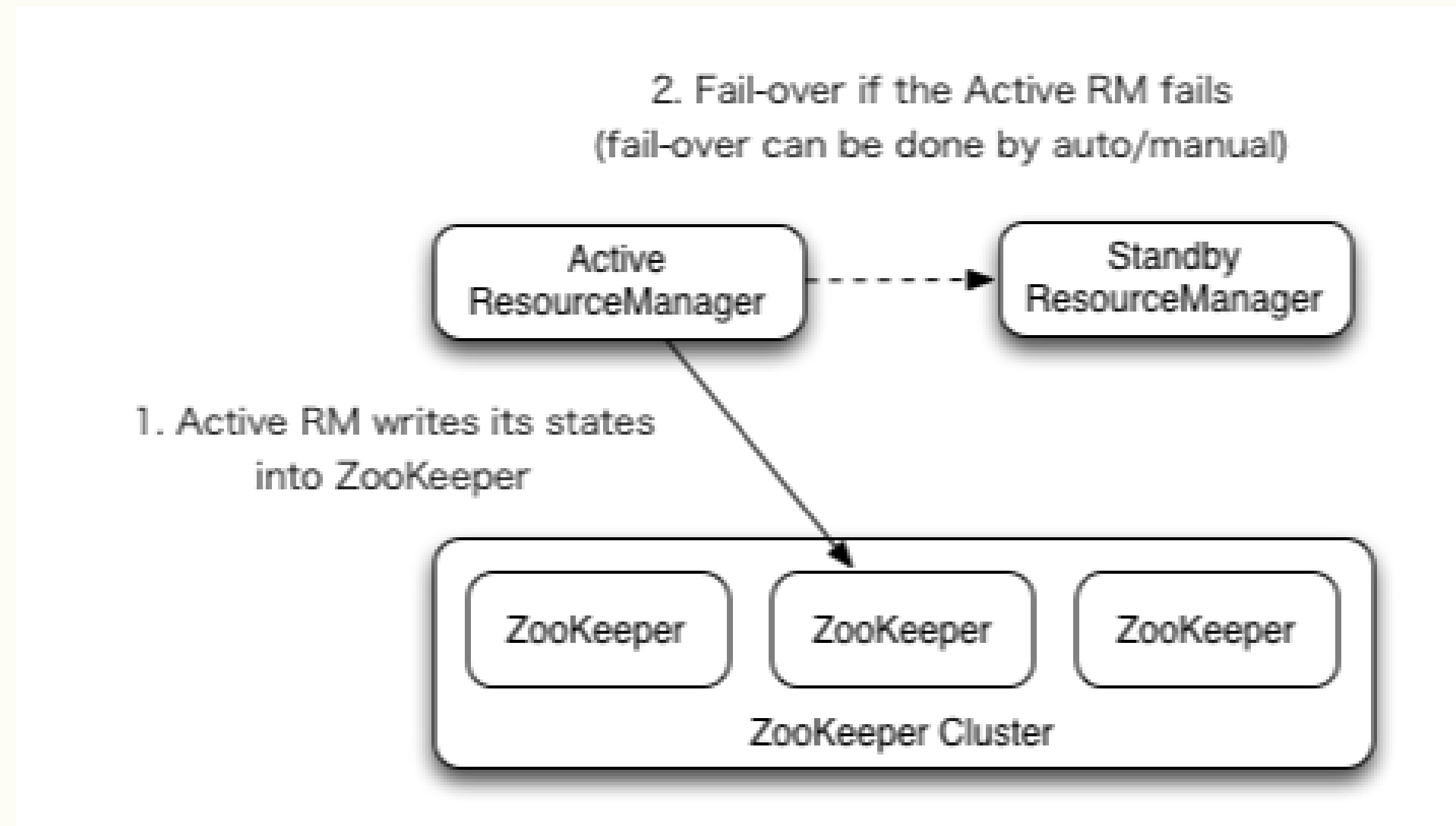
YARN Queue Oluřturma

- Yeni bir kuyruk oluřtururak rnek bir uygulama alıřtırıp resource manager web ui'dan bu kuyruęu inceleyelim.



YARN HA Mimari

- YARN ResourceManager'ın yüksek erişilebilirlik sağlamak için kullanılan temel yöntem, ResourceManager HA (High Availability) mimarisidir. Clusterda aktif ve standby olmak üzere iki resource manager bulunur. Aktif durumda olan kaynak yönetimini sağlarken, standby resource manager ise active node'un arızalanması durumunda onun görevlerini üstlenir.



Soru-Cevap

